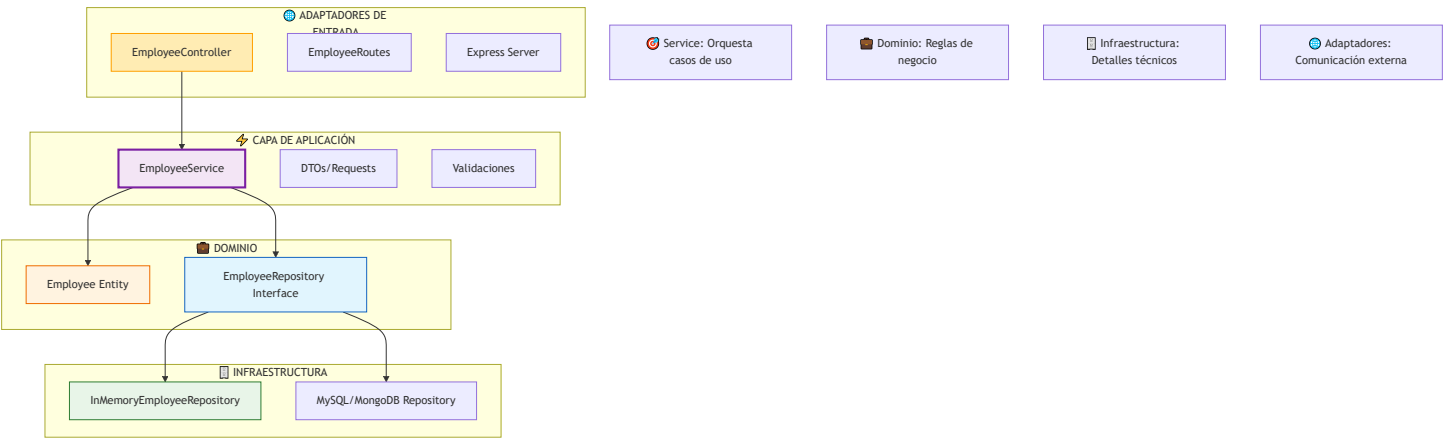


Guía 4: Capa de Aplicación - Service y Casos de Uso

Objetivos de Aprendizaje

Diagrama de Arquitectura Completa



Objetivos por Nivel Cognitivo

Nivel	Verbo	Objetivo	Evidencia
Recordar	Identificar	Componentes de la capa de aplicación	Lista de responsabilidades del Service
Comprender	Explicar	Patrón Service Layer y su propósito	Diagrama de flujo de un caso de uso
Aplicar	Implementar	EmployeeService con casos de uso reales	Código funcional con métodos CRUD
Analizar	Diferenciar	Lógica de aplicación vs lógica de dominio	Tabla comparativa con ejemplos
Evaluar	Validar	Separación correcta de responsabilidades	Checklist de principios SOLID aplicados

Nivel	Verbo	Objetivo	Evidencia
Crear	Diseñar	Sistema completo que sigue arquitectura hexagonal	API funcionando de extremo a extremo

1. ¿Qué es la Capa de Aplicación?

Definición y Ubicación Arquitectónica

Capa de Aplicación: Es la **orquestadora** del sistema. Coordina el flujo entre:

-  **Usuarios** (a través de controladores)
-  **Dominio** (entidades y reglas de negocio)
-  **Infraestructura** (persistencia, servicios externos)

Ubicación en Arquitectura Hexagonal:

```
src/application/
├── services/           #  Services - Casos de uso
├── dtos/              #  DTOs - Objetos de transferencia
└── validations/       #  Validaciones de aplicación
```

Patrón de Diseño: Service Layer

Service Layer es un patrón que:

-  **Coordina** múltiples operaciones del dominio
-  **Orquesta** transacciones y flujos complejos
-  **Maneja** validaciones de aplicación (no de dominio)
-  **Traduce** entre DTOs externos y entidades del dominio
-  **Aplica** políticas de seguridad y auditoría

Analogía: Un **maestro de ceremonias** en un evento:

- No cocina (dominio)
- No limpia (infraestructura)
- Coordina quién hace qué y cuándo

2. Implementando el EmployeeService

2.1 Estructura de Archivos

```
src/application/  
├─ dtos/  
│   ├─ requests/  
│   │   ├─ create-employee.request.ts  
│   │   └─ update-employee.request.ts  
│   └─ responses/  
│       └─ employee.response.ts  
├─ services/  
│   └─ employee.service.ts  
└─ validations/  
    └─ employee.validations.ts
```

2.2 DTOs (Data Transfer Objects)

```
// 📁 src/application/dtos/requests/create-employee.request.ts
/**
 * 📄 DTO para creación de empleados
 * Contiene datos que vienen del exterior (API, CLI, etc.)
 * Se valida en la capa de aplicación
 */
export interface CreateEmployeeRequest {
  cedula: string;
  nombres: string;
  salario: number;
}

// 📁 src/application/dtos/requests/update-employee.request.ts
export interface UpdateEmployeeRequest {
  nombres?: string;
  salario?: number;
}

// 📁 src/application/dtos/responses/employee.response.ts
export interface EmployeeResponse {
  id: string;
  cedula: string;
  nombres: string;
  salario: number;
  createdAt: string;
  updatedAt: string;
}
```

2.3 Implementación del EmployeeService

```
// 📁 src/application/services/employee.service.ts
// 🔄 UBICACIÓN: APLICACIÓN - Orquesta casos de uso

import type { EmployeeRepository } from '../../domain/employee.repository.js';
import { Employee } from '../../domain/employee.entity.js';
import type {
  CreateEmployeeRequest,
  UpdateEmployeeRequest,
  EmployeeResponse
} from '../../dtos/index.js';

/**
 * ⚠️ IMPORTANTE: Este Service está en APLICACIÓN porque:
 *
 * 1. ORQUESTA operaciones complejas
 * 2. COORDINA múltiples componentes del dominio
 * 3. MANEJA validaciones de aplicación (no de dominio)
 * 4. TRADUCE entre DTOs externos y entidades
 * 5. APLICA políticas (seguridad, auditoría, etc.)
 */
export class EmployeeService {
  constructor(private repository: EmployeeRepository) {}

  /**
   * 📄 CASO DE USO: Crear empleado
   * Orquesta: validación → creación → persistencia → respuesta
   */
  async createEmployee(request: CreateEmployeeRequest): Promise<EmployeeResponse> {
    // ✅ VALIDACIÓN DE APLICACIÓN (no de dominio)
    this.validateCreateRequest(request);

    // ✅ VERIFICAR reglas de negocio (a nivel aplicación)
    const exists = await this.repository.existsByCedula(request.cedula);
    if (exists) {
      throw new Error('Ya existe un empleado con esta cédula');
    }

    // ✅ CREAR entidad (dominio hace validaciones internas)
    const employee = Employee.create(
      request.cedula,
      request.nombres,
```

```

        request.salario
    );

    // ✅ PERSISTIR (orquestación con infraestructura)
    await this.repository.save(employee);

    // ✅ TRANSFORMAR a respuesta (traducción DTO)
    return this.toResponse(employee);
}

/**
 * 🔍 CASO DE USO: Obtener empleado por ID
 */
async getEmployeeById(id: string): Promise<EmployeeResponse> {
    const employee = await this.repository.findById(id);

    if (!employee) {
        throw new Error(`Empleado con ID ${id} no encontrado`);
    }

    return this.toResponse(employee);
}

/**
 * 📋 CASO DE USO: Listar todos los empleados
 */
async getAllEmployees(): Promise<EmployeeResponse[]> {
    const employees = await this.repository.findAll();
    return employees.map(emp => this.toResponse(emp));
}

/**
 * ✏️ CASO DE USO: Actualizar empleado
 * Orquesta: búsqueda → validación → actualización → persistencia
 */
async updateEmployee(
    id: string,
    request: UpdateEmployeeRequest
): Promise<EmployeeResponse> {
    // ✅ VALIDAR que hay algo para actualizar
    if (!request.nombres && !request.salario) {
        throw new Error('Debe proporcionar nombres o salario para actualizar');
    }
}

```

```

// ✅ BUSCAR empleado
const employee = await this.repository.findById(id);
if (!employee) {
  throw new Error(`Empleado con ID ${id} no encontrado`);
}

// ✅ APLICAR actualizaciones (orquestrando llamadas al dominio)
if (request.nombres) {
  this.validateNombres(request.nombres);
  employee.changeNombres(request.nombres);
}

if (request.salario !== undefined) {
  this.validateSalario(request.salario);
  employee.changeSalario(request.salario);
}

// ✅ PERSISTIR cambios
await this.repository.update(employee);

return this.toResponse(employee);
}

/**
 * 🗑️ CASO DE USO: Eliminar empleado
 */
async deleteEmployee(id: string): Promise<void> {
  const deleted = await this.repository.delete(id);

  if (!deleted) {
    throw new Error(`Empleado con ID ${id} no encontrado`);
  }
}

/**
 * 💰 CASO DE USO: Aumentar salario
 * Ejemplo de operación de negocio compleja
 */
async increaseSalary(
  id: string,
  percentage: number
): Promise<EmployeeResponse> {

```

```

// ✅ VALIDACIÓN DE APLICACIÓN
if (percentage < 0 || percentage > 100) {
  throw new Error('El porcentaje debe estar entre 0 y 100');
}

// ✅ BUSCAR empleado
const employee = await this.repository.findById(id);
if (!employee) {
  throw new Error(`Empleado con ID ${id} no encontrado`);
}

// ✅ EJECUTAR operación de dominio
employee.increaseSalary(percentage);

// ✅ PERSISTIR cambios
await this.repository.update(employee);

return this.toResponse(employee);
}

/**
 * 📊 CASO DE USO: Obtener estadísticas
 * Ejemplo de orquestación compleja
 */
async getStatistics(): Promise<{
  total: number;
  averageSalary: number;
  highestSalary: number;
  lowestSalary: number;
}> {
  const employees = await this.repository.findAll();

  if (employees.length === 0) {
    return {
      total: 0,
      averageSalary: 0,
      highestSalary: 0,
      lowestSalary: 0
    };
  }

  const salaries = employees.map(e => e.getSalario());
  const totalSalary = salaries.reduce((sum, salary) => sum + salary, 0);

```



```

    return {
      total: employees.length,
      averageSalary: totalSalary / employees.length,
      highestSalary: Math.max(...salaries),
      lowestSalary: Math.min(...salaries)
    };
  }

  /**
   * 🔍 CASO DE USO: Buscar por rango de salario
   */
  async getEmployeesBySalaryRange(
    min: number,
    max: number
  ): Promise<EmployeeResponse[]> {
    if (min < 0 || max < 0 || min > max) {
      throw new Error('Rango de salario inválido');
    }

    const employees = await this.repository.findBySalaryRange(min, max);
    return employees.map(emp => this.toResponse(emp));
  }

  // 🛡️ VALIDACIONES DE APLICACIÓN
  private validateCreateRequest(request: CreateEmployeeRequest): void {
    if (!request.cedula || request.cedula.length < 8) {
      throw new Error('La cédula debe tener al menos 8 caracteres');
    }

    if (!request.nombres || request.nombres.trim().length < 2) {
      throw new Error('Los nombres deben tener al menos 2 caracteres');
    }

    if (request.salario < 0) {
      throw new Error('El salario no puede ser negativo');
    }
  }

  private validateNombres(nombres: string): void {
    if (nombres.trim().length < 2) {
      throw new Error('Los nombres deben tener al menos 2 caracteres');
    }
  }

```

```

}

private validateSalario(salario: number): void {
  if (salario < 0) {
    throw new Error('El salario no puede ser negativo');
  }

  if (salario > 1000000) {
    throw new Error('El salario no puede exceder $1,000,000');
  }
}

// 🔄 MAPPER: Entidad → DTO de respuesta
private toResponse(employee: Employee): EmployeeResponse {
  return {
    id: employee.getId(),
    cedula: employee.getCedula(),
    nombres: employee.getNombres(),
    salario: employee.getSalario(),
    createdAt: employee.getCreatedAt().toISOString(),
    updatedAt: employee.getUpdatedAt().toISOString()
  };
}
}

```

🏆 3. Principios SOLID Aplicados

Diagrama de Principios SOLID en el Service

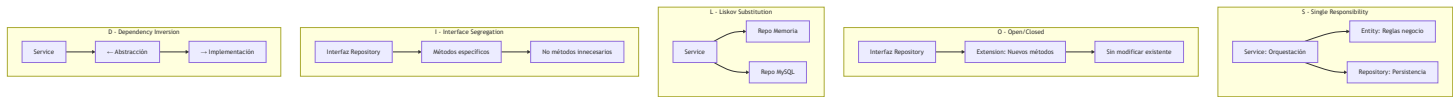


Tabla de Principios SOLID Implementados

Principio	Cómo se aplica en EmployeeService	Beneficio
Single Responsibility	Service solo orquesta, no valida ni persiste directamente	Cambios aislados, más fácil de mantener

Principio	Cómo se aplica en EmployeeService	Beneficio
Open/Closed	Se pueden agregar nuevos casos de uso sin modificar existentes	Extensible sin romper lo que ya funciona
Liskov Substitution	Service funciona con cualquier Repository (memoria, MySQL, etc.)	Intercambiable, fácil de testear
Interface Segregation	Service depende solo de métodos que usa de Repository	Bajo acoplamiento, dependencias claras
Dependency Inversion	Service depende de abstracción (interfaz), no de implementación	Desacoplado, fácil cambiar implementación

Ejemplos en Código:

```
// ✅ SINGLE RESPONSIBILITY
// EmployeeService solo orquesta, no hace validaciones complejas
// ni persiste directamente

// ✅ OPEN/CLOSED
// Podemos agregar getEmployeesByDepartment() sin modificar métodos existentes

// ✅ LISKOV SUBSTITUTION
const service1 = new EmployeeService(new InMemoryRepository());
const service2 = new EmployeeService(new MySQLRepository());
// Ambos funcionan igual

// ✅ INTERFACE SEGREGATION
// Service solo usa métodos de Repository que necesita
// No está obligado a implementar métodos que no usa

// ✅ DEPENDENCY INVERSION
// Service ← RepositoryInterface ← Memory/MySQL/MongoDB
// Depende de abstracción, no de implementación concreta
```



4. Ventajas de esta Separación

Ventaja 1: Testing Efectivo

```
// ✅ Test del Service con mock del Repository
const mockRepository = {
  save: jest.fn(),
  findById: jest.fn().mockResolvedValue(mockEmployee),
  existsByCedula: jest.fn().mockResolvedValue(false)
};

const service = new EmployeeService(mockRepository);

// Test AISLADO del Service
await service.createEmployee({ cedula: '123', nombres: 'Test', salario: 1000 });
expect(mockRepository.save).toHaveBeenCalledTimes(1);
```

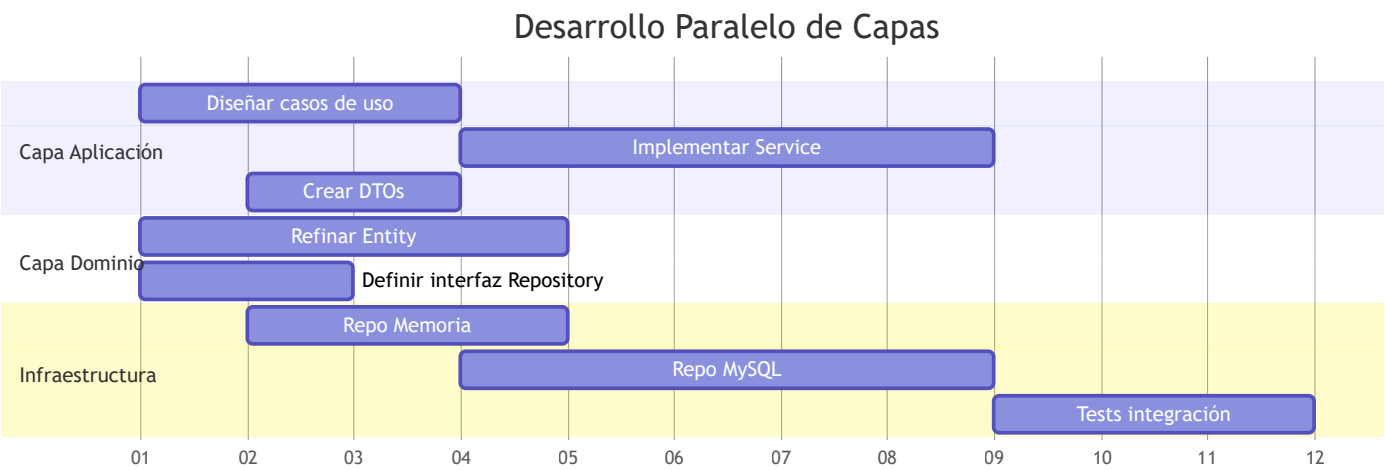
Ventaja 2: Cambios Tecnológicos sin Dolor

```
// ✅ Cambiar de memoria a MySQL en UNA línea
// Desarrollo
const devService = new EmployeeService(new InMemoryEmployeeRepository());

// Producción
const prodService = new EmployeeService(new MySQLEmployeeRepository(config));

// ✅ El código del Service NO CAMBIA
```

Ventaja 3: Desarrollo en Paralelo



Ventaja 4: Mantenibilidad a Largo Plazo

```
// AÑO 1: Sistema básico
service.createEmployee({...});

// AÑO 2: Agregamos validación adicional
// Solo modificamos validateCreateRequest()
// No tocamos dominio ni infraestructura

// AÑO 3: Agregamos logging de auditoría
// Solo agregamos this.auditLog() en Service
// No afecta otras capas

// AÑO 4: Cambiamos de MySQL a MongoDB
// Solo cambiamos new MySQLRepository() por new MongoDBRepository()
// Service y Dominio intactos
```

Ventaja 5: Separación Clara de Preocupaciones

Capa	Responsabilidad	Ejemplo
Aplicación	Orquestrar flujos, validar DTOs	validateCreateRequest()
Dominio	Reglas de negocio puras	employee.changeSalario()
Infraestructura	Detalles técnicos	repository.save()



5. Pruebas del EmployeeService

5.1 Archivo de Pruebas

```
// 📁 src/test-service.ts
(async () => {
  console.log('🔧 PROBANDO EMPLOYEE SERVICE');
  console.log('=====\\n');

  const { EmployeeService } = await import('./application/services/employee.service.js');
  const { InMemoryEmployeeRepository } = await import('./infrastructure/repositories/in-memory.i

  const repository = new InMemoryEmployeeRepository();
  const service = new EmployeeService(repository);

  try {
    // 📌 PRUEBA 1: Crear empleado
    console.log('1. 📄 Crear empleado');
    const response = await service.createEmployee({
      cedula: '1234567890',
      nombres: 'Juan Pérez',
      salario: 2500
    });
    console.log(`✅ Creado: ${response.nombres} - ID: ${response.id}\\n`);

    // 📌 PRUEBA 2: Obtener por ID
    console.log('2. 🔍 Obtener empleado por ID');
    const found = await service.getEmployeeById(response.id);
    console.log(`✅ Encontrado: ${found.nombres} - Salario: ${found.salario}\\n`);

    // 📌 PRUEBA 3: Actualizar empleado
    console.log('3. ✎ Actualizar empleado');
    const updated = await service.updateEmployee(response.id, {
      nombres: 'Juan Carlos Pérez',
      salario: 2800
    });
    console.log(`✅ Actualizado: ${updated.nombres} - Nuevo salario: ${updated.salario}\\n`);

    // 📌 PRUEBA 4: Aumentar salario
    console.log('4. 💰 Aumentar salario 15%');
    const withIncrease = await service.increaseSalary(response.id, 15);
    console.log(`✅ Salario aumentado: ${withIncrease.salario.toFixed(2)}\\n`);
```

```

// 📌 PRUEBA 5: Obtener todos
console.log('5. 📄 Obtener todos los empleados');
const allEmployees = await service.getAllEmployees();
console.log(`✅ Total empleados: ${allEmployees.length}\n`);

// 📌 PRUEBA 6: Estadísticas
console.log('6. 📊 Obtener estadísticas');
const stats = await service.getStatistics();
console.log(`✅ Estadísticas:`);
console.log(`    • Total: ${stats.total}`);
console.log(`    • Salario promedio: ${stats.averageSalary.toFixed(2)}`);
console.log(`    • Salario más alto: ${stats.highestSalary.toFixed(2)}`);
console.log(`    • Salario más bajo: ${stats.lowestSalary.toFixed(2)}\n`);

// 📌 PRUEBA 7: Buscar por rango
console.log('7. 🔍 Buscar por rango de salario ($2500-$3000)');
const inRange = await service.getEmployeesBySalaryRange(2500, 3000);
console.log(`✅ En rango: ${inRange.length} empleados\n`);

// 📌 PRUEBA 8: Eliminar empleado
console.log('8. 🗑 Eliminar empleado');
await service.deleteEmployee(response.id);
console.log(`✅ Empleado eliminado\n`);

console.log('🎉 ¡TODAS LAS PRUEBAS DEL SERVICE PASARON!');

} catch (error: any) {
  console.error('❌ ERROR:', error.message);
}
})();

```


5.2 Actualizar Scripts npm

```
// 📁 package.json
{
  "scripts": {
    "dev": "nodemon src/app.ts",
    "build": "tsc",
    "start": "node dist/app.js",
    "test:entity": "npx tsc && node dist/test-employee.js",
    "test:repository": "npx tsc && node dist/test-repository.js",
    "test:service": "npx tsc && node dist/test-service.js",
    "test:all": "npm run test:entity && npm run test:repository && npm run test:service"
  }
}
```

```
# Ejecutar pruebas
npm run test:service
```

6. Ejercicios de Profundización

Ejercicio 1: Agregar Nuevo Caso de Uso

```
// Agrega este método a EmployeeService:

/**
 *  CASO DE USO: Transferir empleado a nuevo departamento
 *
 * Requisitos:
 * 1. Validar que el departamento exista
 * 2. Verificar que el empleado no esté ya en ese departamento
 * 3. Registrar fecha de transferencia
 * 4. Notificar a RRHH (simulado)
 * 5. Retornar empleado actualizado
 */
async transferEmployee(
  employeeId: string,
  departmentId: string
): Promise<EmployeeResponse> {
  // Implementa este caso de uso
  // Muestra cómo orquestas múltiples operaciones
}
```

Ejercicio 2: Agregar Validaciones Avanzadas

```
// Crea un ValidationService que:  
// 1. Valide formato de cédula según país  
// 2. Valide que nombres no contengan caracteres especiales  
// 3. Valide rangos de salario según posición  
// 4. Reutilice validaciones en múltiples servicios  
  
class EmployeeValidationService {  
    validateCedula(cedula: string): void {  
        // Validar según país  
    }  
  
    validateSalaryForPosition(position: string, salary: number): void {  
        // Rangos según posición  
    }  
}
```

7. Checklist de Completitud

Para el Estudiante:

- ☐ Entendí la diferencia entre capa de aplicación y capa de dominio
- ☐ Implementé EmployeeService en src/application/services/
- ☐ Creé DTOs para request/response en src/application/dtos/
- ☐ Apliqué principios SOLID en el diseño del Service
- ☐ Implementé casos de uso reales (CRUD + operaciones especiales)
- ☐ Creé archivo de pruebas src/test-service.ts
- ☐ Ejecuté pruebas exitosamente con npm run test:service
- ☐ Comprendí el patrón Service Layer y sus ventajas

Para Evaluación del Profesor:

- ☐ **Separación correcta:** Service orquesta, no implementa lógica de negocio
- ☐ **SOLID aplicado:** Dependency Injection, Single Responsibility, etc.
- ☐ **DTOs implementados:** Separación clara entre entidades y objetos de transferencia
- ☐ **Validaciones apropiadas:** En aplicación (no en dominio donde no corresponden)

- ☐ **Pruebas completas:** Cobertura de todos los casos de uso
- ☐ **Arquitectura hexagonal:** Service depende de abstracciones, no implementaciones



8. Próximos Pasos

Guía 5 - Adaptadores de Entrada (Controller + Routes):

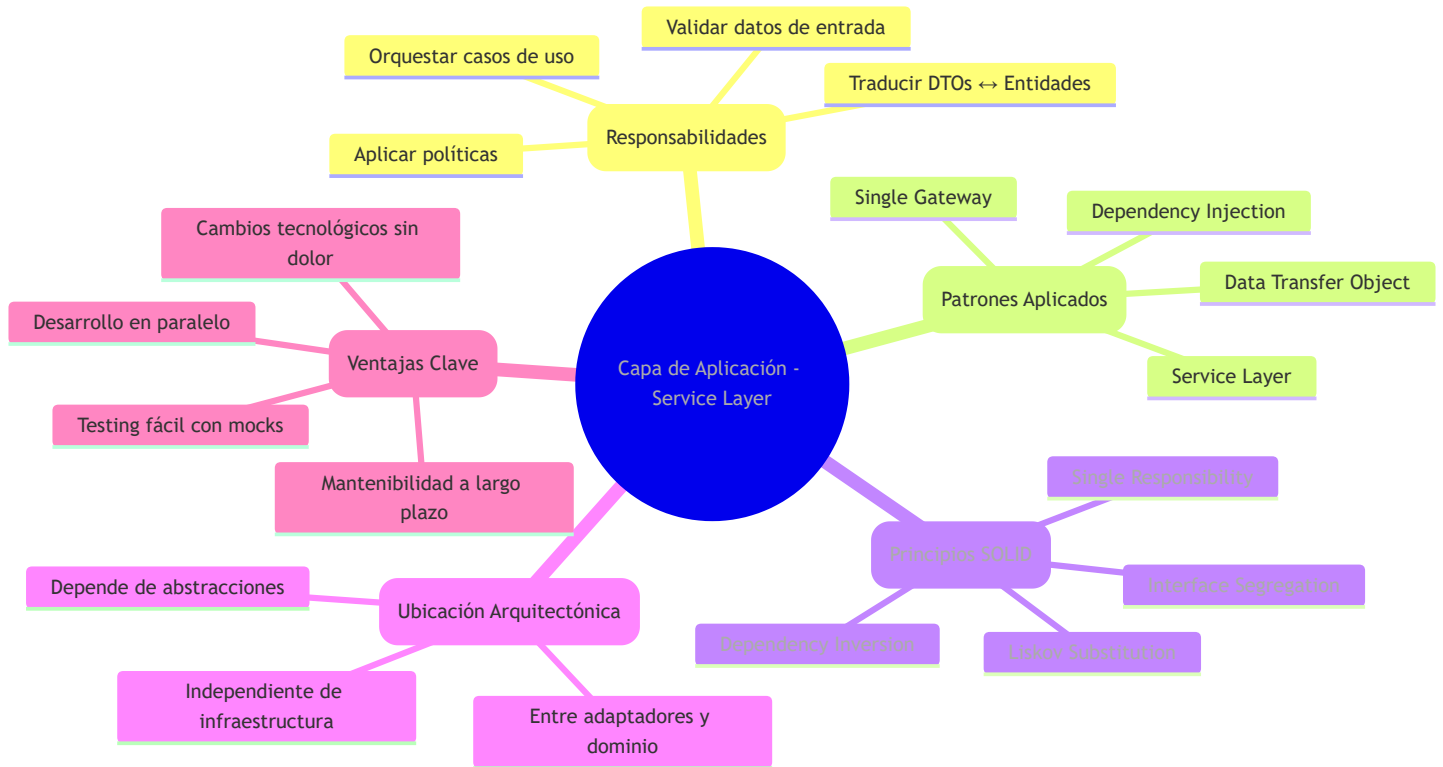
- Crear `EmployeeController` con endpoints REST
- Implementar `EmployeeRoutes` con Express
- Conectar Controller → Service → Repository → Entity
- Probar API completa con Postman/curl

Tarea Preparatoria:

1. Ejecutar `npm run test:all` y verificar que todo funciona
2. Pensar en endpoints REST para cada caso de uso
3. Investigar sobre middleware en Express (error handling, validation)



Resumen de lo Aprendido



✓ Has completado la Guía 4:

- ✓ Entendiste el propósito de la capa de aplicación
- ✓ Implementaste el patrón Service Layer correctamente
- ✓ Aplicaste principios SOLID en el diseño
- ✓ Creaste un Service que orquesta casos de uso reales
- ✓ Separaste responsabilidades entre aplicación, dominio e infraestructura
- ✓ Probaste el Service de forma aislada y completa